

P R O J E C T P L A N

LINKLY

Upload it once. Get a link. Share it anywhere.

React + Vite

Laravel REST API

MySQL

Dark Glassmorphism UI

External API Integrations

FULL-STACK FILE SHARING PLATFORM · TECHNICAL BLUEPRINT

Table of Contents

01	What is LINKLY?	Overview & objectives
02	Technology Stack	Frontend, backend, DB, APIs
03	System Architecture	How everything talks to each other
04	The API Layer (core focus)	Internal REST API + external APIs
05	Database Design	Tables & relationships
06	UI Direction — Dark Glassmorphism	Colors, blur, type, motion
07	Screen-by-screen Design Notes	Upload, share page, dashboard, admin
08	React Frontend Structure	Full folder tree
09	Laravel Backend Structure	Full folder tree
10	Feature Tiers	MVP → advanced → signature features
11	Security Checklist	What we actually implement
12	Development Schedule	14-day build plan
13	Demo Script	How we present it

What is LINKLY?

LINKLY is a web-based file-sharing platform. A user uploads a file — image, video, PDF, document, or archive — and instantly gets a unique shareable link (linkly.app/f/8xK92LmP). Anyone who opens that link sees a live preview of the file and can download it, no account required. Registered users get a dashboard to manage their files, track downloads, set link expiration, and more. Admins get a full control panel.

Core objectives

1. Let users upload files through a polished web interface, with drag-and-drop and paste-to-upload.
2. Store files securely and record file metadata in MySQL.
3. Generate a unique, unguessable share token for every upload.
4. Let anyone with the link preview and download the file — no login wall.
5. Give registered users a dashboard: manage files, see download counts, delete, set expiration.
6. Give admins a control panel: users, files, storage, platform-wide stats.
7. **Integrate real external APIs** so the project demonstrates genuine third-party API consumption, not just an internal CRUD API talking to itself (see Chapter 4 — this is the centerpiece).
8. Implement real security: validation, auth, rate limiting, expiring links, hashed passwords.
9. Ship a UI that looks and feels like a native iOS app — dark, glassy, animated, not a bootstrap form.

In one line

"Upload it once. Get a link. Share it anywhere." — the whole product pitch fits in a sentence, which is exactly what a good demo needs.

Technology Stack

Frontend

- React 18
- Vite
- React Router
- Axios
- Framer Motion
- Tailwind CSS
- Zustand

React handles every screen: auth, upload, dashboard, public share page, admin panel. **Framer Motion** drives the spring/iOS-style animations. **Zustand** (tiny state library) holds auth + upload state without the ceremony of Redux.

Backend

- Laravel 11
- Sanctum
- Laravel Queues
- Intervention/Image
- Spatie MediaLibrary (optional)

Laravel is the entire REST API: auth (Sanctum tokens), file validation & storage, share-token generation, download handling, admin endpoints, and — importantly — the layer that calls out to **external APIs** (virus scanning, geolocation) via queued jobs so uploads never feel slow.

Database

- MySQL 8

Stores users, files, download logs, collections, and notifications. See Chapter 5 for the full schema.

Storage

- Laravel Filesystem
- Local disk (dev)
- S3-compatible (prod-ready)

Local disk for development; the storage config is driver-agnostic so switching to S3 / Spaces / R2 later is a config change, not a rewrite.

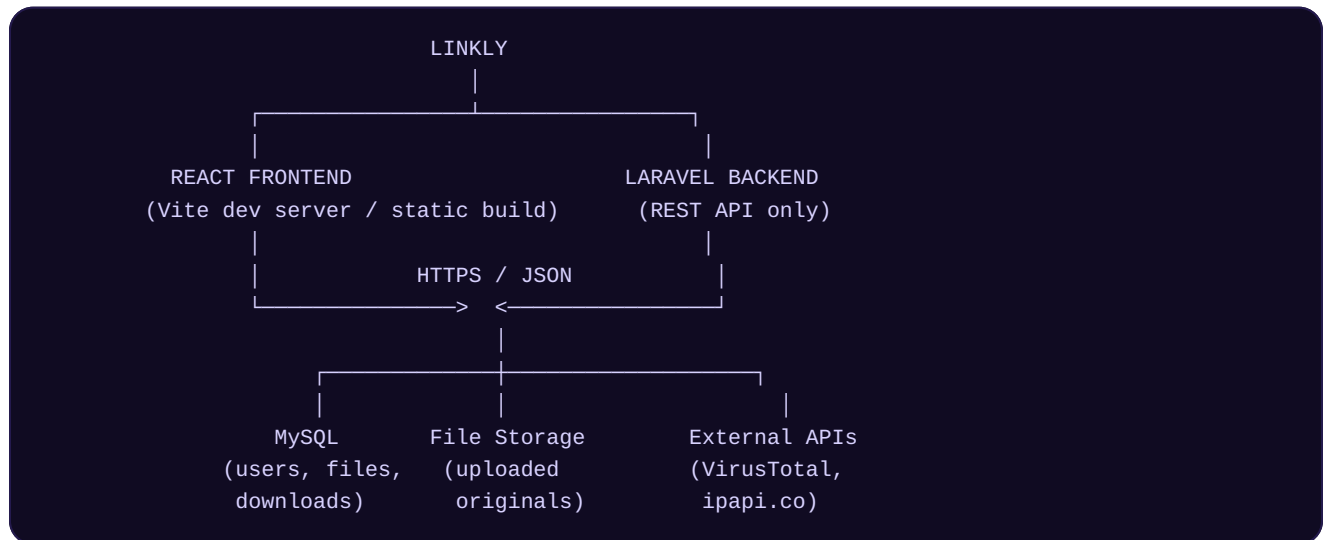
External APIs (this is what makes it a real "API project")

An internal REST API alone is just CRUD. Two genuine third-party integrations are what turn LINKLY into a project with real API-consumption depth — both are free-tier friendly and directly support features already in the plan (security + analytics), so they're not bolted on for show:

API	Used for	Why it's a legitimate fit
VirusTotal API	Malware / safety scan on every upload	Laravel queues a job that hashes the file and checks it against VirusTotal after upload. Ties directly into the "security" pillar of the project — instead of just claiming the platform is safe, it's demonstrably scanned.
ipapi.co (or ip-api.com)	Geolocation on each download	When a file is downloaded, Laravel logs the requester's approximate country from their IP. Powers the admin "download world map" statistic — a genuinely more interesting chart than a plain counter.

Both calls happen **server-side, asynchronously** (Laravel queued jobs), never blocking the upload/download response. This is a good talking point for the presentation: "the API integration doesn't slow the user down — it happens in the background."

System Architecture



React **never** talks to MySQL or the filesystem directly. Every action — upload, list files, generate a link, download, admin action — goes through a Laravel API endpoint. This one rule keeps the whole system's security boundary in one place (Laravel) instead of scattered across the frontend.

Request lifecycle for an upload

1. User drops a file on the React upload zone.
2. React `POST /api/files` with the file as multipart form data.
3. Laravel validates type/size, stores it, writes a `files` row, generates a share token.
4. Laravel **dispatches a queued job** to hash the file and check it against the VirusTotal API.
5. Laravel responds immediately with the file record + share link — the user isn't kept waiting on the scan.
6. React shows the "Upload complete" card with the link, QR code, and copy button.
7. Once the scan job finishes, the file's `scan_status` flips to `clean` or `flagged` (polled or pushed via a lightweight status endpoint).

The API Layer (core focus of the project)

This is the section to lean on hardest in the write-up and the demo — it's the actual engineering content of the project.

4.1 — Authentication

Method	Endpoint	Description
POST	/api/register	Create account (name, email, password)
POST	/api/login	Returns a Sanctum bearer token
POST	/api/logout	Revokes the current token
GET	/api/user	Returns the authenticated user

4.2 — Files (authenticated)

Method	Endpoint	Description
GET	/api/files	List the current user's files (paginated, sortable)
POST	/api/files	Upload a new file → returns file + share_token
GET	/api/files/{id}	File details + download/scan status
PATCH	/api/files/{id}	Update expiration, password protection, custom slug
DELETE	/api/files/{id}	Delete a file (owner-only, enforced server-side)

4.3 — Public sharing (no auth required)

Method	Endpoint	Description
GET	/api/share/{token}	Public metadata for the share page (name, size, type, preview URL)
POST	/api/share/{token}/unlock	Submit password for protected links
GET	/api/share/{token}/download	Streams the file, increments download count, logs geolocation

4.4 — Admin (role-gated)

Method	Endpoint	Description
GET	/api/admin/users	List / search all users
DELETE	/api/admin/users/{id}	Suspend or delete a user
GET	/api/admin/files	List / search all files platform-wide

DELETE	/api/admin/files/{id}	Force-delete any file
GET	/api/admin/statistics	Totals + time-series for the dashboard charts

4.5 — Example: upload response

```
{
  "id": 4821,
  "name": "project.pdf",
  "size": 4200000,
  "type": "application/pdf",
  "share_token": "8xK92LmP",
  "share_url": "https://linkly.app/f/8xK92LmP",
  "scan_status": "pending",
  "expires_at": null,
  "created_at": "2026-09-11T10:42:00Z"
}
```

4.6 — External API call (VirusTotal), server-side only

```
// app/Jobs/ScanUploadedFile.php (simplified)
$hash = hash_file('sha256', $file->path);

$response = Http::withHeaders([
  'x-apikey' => config('services.virustotal.key'),
])->get("https://www.virustotal.com/api/v3/files/{$hash}");

$file->update([
  'scan_status' => $response->ok() ? 'clean' : 'flagged',
]);
```

The VirusTotal / ipapi API keys live only in Laravel's `.env` — React never sees them. All third-party calls are proxied through Laravel, which is both the secure pattern and the one worth explaining explicitly in the report ("why doesn't the frontend call these APIs directly?").

Database Design

users

Column	Type	Notes
id	bigint	PK
name	varchar	
email	varchar, unique	
password	varchar	hashed (bcrypt)
role	enum	<code>user</code> / <code>admin</code>
created_at / updated_at	timestamp	

files

Column	Type	Notes
id	bigint	PK
user_id	bigint, nullable	FK → users (nullable = anonymous upload, if allowed)
original_name	varchar	
stored_name	varchar	random name on disk, never trust the original
file_path	varchar	
file_type	varchar	MIME type
file_size	bigint	bytes
share_token	varchar, unique	random 8–12 char slug
custom_slug	varchar, nullable	optional vanity link
password_hash	varchar, nullable	optional link password
scan_status	enum	<code>pending</code> / <code>clean</code> / <code>flagged</code>
downloads	int	counter
burn_after_read	boolean	delete after first download
expires_at	timestamp, nullable	
created_at / updated_at	timestamp	

download_logs

Column	Type	Notes
--------	------	-------

id	bigint	PK
file_id	bigint	FK → files
ip_address	varchar	hashed/truncated for privacy if needed
country	varchar, nullable	resolved via ipapi.co
downloaded_at	timestamp	

collections (stretch feature — bundle multiple files under one link)

Column	Type	Notes
id	bigint	PK
user_id	bigint	FK → users
title	varchar	
share_token	varchar, unique	

```

users (1) ———< (many) files
files (1) ———< (many) download_logs
users (1) ———< (many) collections ———< (many) files

```

UI Direction — Dark Glassmorphism, "iPhone" Feel

The whole interface should feel like iOS Control Center / Dynamic Island: near-black backgrounds, frosted glass cards, big soft shadows, spring-based motion — never flat, never sharp, never instant.

Color palette

Token	Value	Usage
--bg-base	#07060c	App background
--bg-gradient	radial-gradient(circle at top, #1b1338, #07060c 70%)	Ambient glow behind content
--glass-fill	rgba(255,255,255,0.06)	Card backgrounds
--glass-border	rgba(255,255,255,0.14)	1px card borders
--accent-violet	#7c5cff	Primary buttons, links, active states
--accent-cyan	#4fd8ff	Secondary accent, progress, success glow
--text-primary	#f5f3ff	Headings
--text-muted	rgba(245,243,255,0.6)	Body / secondary text

Glass card recipe (CSS)

```
.glass-card {
  background: rgba(255, 255, 255, 0.06);
  border: 1px solid rgba(255, 255, 255, 0.14);
  border-radius: 24px;
  backdrop-filter: blur(20px) saturate(160%);
  -webkit-backdrop-filter: blur(20px) saturate(160%);
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.45);
}
```

Typography

- **Font:** "Inter" or "SF Pro Display" fallback — rounded, geometric, very iOS.
- Big, confident headlines (32–48px) on the landing/upload screen; tight tracking on all-caps labels.
- Numbers (download counts, storage stats) get a slightly heavier weight and tabular-nums for that "widget" feel.

Motion (Framer Motion)

Interaction	Animation
Page/route transition	Fade + slight upward slide, spring easing (mass: 1, stiffness: 210, damping: 24)
Button press	Scale to 0.96 on tap, spring back — mimics iOS button haptics visually
Card appear (file list)	Stagger children 40ms apart, fade + scale from 0.95 → 1

Upload progress	A pill that morphs/grows like a Dynamic Island — expands from a small capsule to a full status card
Modal open (delete confirm, password unlock)	Backdrop blur fades in, sheet springs up from bottom (iOS action sheet style)
Success state	Soft cyan glow pulse + checkmark draw-on animation

Signature "iPhone-like" touches

- **Dynamic-Island upload indicator:** a small dark capsule pinned top-center that expands to show upload progress, then contracts to a checkmark — instead of a plain progress bar.
- **Frosted top nav bar** that blurs the content scrolling beneath it, exactly like iOS navigation bars.
- **Bottom sheet modals** instead of centered popups for delete confirmations, share options, password prompts.
- **Soft haptic-style micro-interactions:** every clickable element scales down slightly on press.
- **Segmented control** (pill-shaped tab switcher) for things like "My Files / Shared / Collections" instead of plain tabs.

Screen-by-screen Design Notes

Landing / Upload

Full-viewport dark gradient background. Centered glass card with a large dashed drop-zone. Dragging a file over the *whole page* triggers a fullscreen blurred overlay ("Drop it anywhere"). Paste-to-upload (Ctrl/Cmd+V) works from anywhere on the page.

Upload complete

The Dynamic-Island capsule contracts into a glass card showing: file icon, name, size, the share link, a **QR code** (scan-to-download in the demo), a copy button with a satisfying checkmark micro-animation, and expiration / burn-after-read toggles.

Public share page

No nav bar, no chrome — just the file. A large glass card centered on the dark gradient background showing a real preview (image thumbnail, inline video player, PDF first-page render, or a generic icon for archives), file size, scan status badge (Clean  from VirusTotal), and a big glowing Download button.

Dashboard ("My Files")

Segmented control at top: *My Files* / *Shared* / *Collections*. Each file is a glass row with a live download counter, quick actions (share, copy link, delete) that appear on hover, and a search bar with instant filtering.

Admin panel

Same dark-glass language, denser data: KPI cards up top (users, files, downloads, storage — each in its own glass tile with a subtle glow), a table of recent uploads, and the download-geolocation world map as the centerpiece stat.

React Frontend — Folder Structure

```

frontend/
├── public/
│   └── favicon.svg
├── src/
│   ├── main.jsx                # app entry, mounts
│   ├── App.jsx                 # router + global providers
│   ├── index.css               # Tailwind base + design tokens
│   ├──
│   ├── assets/
│   │   ├── icons/             # file-type icons (pdf, zip, mp4...)
│   │   └── illustrations/
│   ├──
│   ├── styles/
│   │   └── tokens.css          # --bg-base, --glass-fill, etc.
│   ├──
│   ├── components/
│   │   ├── ui/
│   │   │   ├── GlassCard.jsx
│   │   │   ├── Button.jsx
│   │   │   ├── BottomSheet.jsx
│   │   │   ├── SegmentedControl.jsx
│   │   │   ├── Badge.jsx
│   │   │   └── Toast.jsx
│   │   ├── upload/
│   │   │   ├── DropZone.jsx
│   │   │   ├── DynamicIslandProgress.jsx
│   │   │   ├── PasteListener.jsx
│   │   │   └── UploadCompleteCard.jsx
│   │   ├── files/
│   │   │   ├── FileRow.jsx
│   │   │   ├── FilePreview.jsx    # image/video/pdf/generic switch
│   │   │   ├── QRCodeBlock.jsx
│   │   │   └── ExpirationPicker.jsx
│   │   ├── admin/
│   │   │   ├── StatCard.jsx
│   │   │   ├── DownloadsMap.jsx
│   │   │   └── UsersTable.jsx
│   │   └── layout/
│   │       ├── NavBar.jsx        # frosted, blurs content on scroll
│   │       ├── AuthLayout.jsx
│   │       └── DashboardLayout.jsx
│   ├──
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Login.jsx
│   │   ├── Register.jsx
│   │   ├── Dashboard.jsx
│   │   ├── Upload.jsx
│   │   ├── FileDetails.jsx
│   │   ├── SharedFile.jsx        # public /f/:token page
│   │   ├── Profile.jsx
│   │   └── admin/
│   │       └── AdminDashboard.jsx

```

```
|
|
|   └─ AdminUsers.jsx
|   └─ AdminFiles.jsx
|   └─ AdminStatistics.jsx
|
| └─ services/
|   └─ api.js           # axios instance, base URL, interceptors
|   └─ authService.js
|   └─ fileService.js
|   └─ adminService.js
|
| └─ hooks/
|   └─ useAuth.js
|   └─ useUpload.js
|   └─ useDebounce.js
|
| └─ context/
|   └─ AuthContext.jsx
|
| └─ store/
|   └─ uploadStore.js   # zustand store
|
| └─ utils/
|   └─ formatBytes.js
|   └─ fileIcons.js
|   └─ validators.js
|
| └─ index.html
| └─ tailwind.config.js
| └─ vite.config.js
| └─ package.json
```

Laravel Backend — Folder Structure

```

backend/
├── app/
│   ├── Models/
│   │   ├── User.php
│   │   ├── File.php
│   │   ├── DownloadLog.php
│   │   └── Collection.php
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Api/
│   │   │   │   ├── AuthController.php
│   │   │   │   ├── FileController.php
│   │   │   │   ├── ShareController.php      # public /share/{token}
│   │   │   │   └── Admin/
│   │   │   │       ├── UserController.php
│   │   │   │       ├── FileController.php
│   │   │   │       └── StatisticsController.php
│   │   ├── Requests/
│   │   │   ├── StoreFileRequest.php        # validation: type, size
│   │   │   ├── UpdateFileRequest.php
│   │   │   └── UnlockShareRequest.php
│   │   ├── Resources/
│   │   │   ├── FileResource.php            # shapes JSON responses
│   │   │   └── UserResource.php
│   │   └── Middleware/
│   │       ├── EnsureUserIsAdmin.php
│   │       └── ThrottleDownloads.php
│   ├── Services/
│   │   ├── ShareTokenGenerator.php
│   │   ├── FileStorageService.php
│   │   ├── VirusTotalService.php          # wraps the external API
│   │   └── GeoLocationService.php         # wraps ipapi.co
│   ├── Jobs/
│   │   ├── ScanUploadedFile.php          # queued VirusTotal check
│   │   └── ResolveDownloadLocation.php    # queued ipapi.co lookup
│   ├── Policies/
│   │   └── FilePolicy.php                # "can this user delete this file?"
│   └── Providers/
│       └── AppServiceProvider.php
├── database/
│   ├── migrations/
│   │   ├── create_users_table.php
│   │   ├── create_files_table.php
│   │   └── create_download_logs_table.php

```

```
| | | create_collections_table.php
| | | seeders/
| | | | UserSeeder.php
| | | | FileSeeder.php
| |
| | routes/
| | | api.php
| |
| | config/
| | | services.php # virustotal + ipapi keys
| |
| | storage/
| | | app/uploads/
| |
| | .env
```

Feature Tiers — MVP → Advanced → Signature

● MVP (build this first, get it fully working)

Register → Login → Upload → Store → Generate link → Share → View → Download

● Full version (what you actually present)

- Everything in MVP, plus:
- Dashboard with file management, search, sort, filter
- Link expiration + password-protected links
- Download tracking (with geolocation)
- VirusTotal scan status on every file
- Admin dashboard: users, files, storage, statistics
- Rate limiting, hashed passwords, authorization policies

✨ Signature touches (cheap, high demo impact)

QR code on share page

Scan-to-download live in the demo. Near-zero backend cost.

Burn-after-read links

"Delete after first download" toggle — memorable, easy DB flag.

Dynamic-Island upload progress

Signature iOS-style motion moment for the demo.

Real file preview cards

Actual thumbnails/PDF-render instead of generic icons.

● Stretch (only if time remains)

- Collections — bundle multiple files behind one link
- Client-side encryption (Web Crypto API) before upload, key kept only in the URL fragment
- Custom vanity slugs for logged-in users
- Branded / white-labeled share pages

Security Checklist

Area	Implementation
Authentication	Laravel Sanctum tokens; only authenticated users manage their own files
Authorization	Laravel Policies — a user cannot edit/delete another user's file, enforced server-side
File validation	Allow-list of MIME types + max size (e.g. 100MB), rejected server-side regardless of frontend checks
Unguessable links	Cryptographically random share tokens (not sequential IDs)
Malware scanning	Every upload hashed and checked against VirusTotal before being marked "clean"
Expiring links	Files become inaccessible after their <code>expires_at</code> timestamp
Secure downloads	Files streamed through Laravel — storage paths never exposed to the client
Rate limiting	Laravel throttle middleware on upload/download/auth endpoints
Password hashing	bcrypt for user accounts; separate hashed password for protected links

14-Day Development Schedule

1	Project setup Laravel + React scaffolding, Tailwind + design tokens, React ↔ Laravel connectivity check.
2	Database + auth Migrations, Sanctum auth, register/login screens.
3	Upload core Drop-zone UI, upload endpoint, file storage, share-token generation.
4	Share page Public /f/:token page with preview + download endpoint.
5	Dashboard My Files list, search/sort/filter, delete.
6	External API #1 VirusTotal scan job wired into the upload flow + scan-status badge in UI.
7	External API #2 ipapi.co geolocation on download, download_logs table populated.
8	Expiration + password links Backend logic + UI toggles, burn-after-read flag.
9	Admin dashboard Users/files tables, KPI cards, downloads world map.
10	Security pass Policies, rate limiting, validation edge cases.
11	Signature UI polish Dynamic-Island upload progress, QR codes, bottom sheets, motion pass.
12	Responsive + empty/error states Mobile pass, loading skeletons, expired/flagged states.
13	Testing Auth, upload, download, sharing, expiration, admin, mobile.
14	Docs + presentation API docs, ER diagram, architecture diagram, screenshots, demo run-through.

If something runs long, protect Days 1–5 (the MVP) above everything else — the signature UI polish and stretch features are the first things to trim, not auth/upload/share/download.

Demo Script

1. "This is LINKLY — a secure, glass-UI file-sharing platform." (*Open the dark, animated homepage.*)
2. Drag a PDF onto the page — the fullscreen drop overlay appears, then the Dynamic-Island capsule shows upload progress.
3. "Laravel validates and stores the file, generates a unique token, and queues a background scan against the VirusTotal API."
4. Show the completed upload card: link, QR code, scan status.
5. Open the link in a second browser (or scan the QR code on a phone) — "the recipient needs no account."
6. Click download — switch back to the dashboard: "the download was just logged, including the requester's approximate location, resolved through a second external API."
7. Open the admin panel: KPIs, recent uploads, the downloads world map.
8. Close with: "React and Laravel communicate purely through a documented REST API, and two real external services — VirusTotal and ipapi.co — are integrated server-side, asynchronously, so the user is never kept waiting on them."